HTTP/1.1 from the RFCs: Persistent, Non-Persistent and Pipelined Connections, Conditional GET, and a Caching Proxy

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 Message Formats **4**

3 System Architecture **5**

4 Implementation **5**

4.1 Connections: Persistence, Pipelining and Head-of-Line Blocking 5

4.2 Page-Load Strategies and Their Formulas 5

4.3 The Caching Proxy 6

4.4 State, Authentication and Other Features 7

5 Experiments, Verification and Results **8**

5.1 Running the Module 8

5.2 Page-Load Measurements 8

5.3 The Proxy 9

5.4 Automated Verification 10

6 Conclusion **10**

AI Acknowledgement **10**

1 Introduction

The **HyperText Transfer Protocol** (**HTTP**) is the request/response protocol of the web. A client opens a TCP connection to a server (port 80), sends a **request** (a method, a path, header fields and an optional body), and receives a **response** (a status code, header fields and usually a body). The current HTTP/1.1 specification is split into **RFC 9110** (semantics), **RFC 9111** (caching) and **RFC 9112** (message syntax and connection management); cookies are defined in **RFC 6265**.

A web page is rarely one object: a base HTML file references images, scripts and style sheets, each fetched with its own request. How those requests share TCP connections therefore decides how long a page takes to load:

- **Non-persistent** (the HTTP/1.0 default): one TCP connection per object, so every object costs a handshake round trip plus a request round trip.
- **Persistent** (the HTTP/1.1 default, *keep-alive*): one connection for all objects, one round trip per object.
- **Pipelined**: the client sends all requests back to back without waiting; the responses return in order, so the whole page costs about one extra round trip.

HTTP is stateless; **cookies** add state. Unchanged content does not have to be transferred again thanks to **conditional GET** (a `304 Not Modified` reply) and **caching proxies**.

1.1 What Was Built

Everything in this module is written from the RFCs in pure Java 11 with no external libraries:

1. **An HTTP/1.1 origin server** with two virtual hosts, persistent and non-persistent connections, pipelining, conditional GET, byte ranges, chunked transfer coding, `Expect: 100-continue`, gzip, cookies, Basic authentication, redirects and a full set of status codes.
2. **A browser-like client** that loads a page with four connection strategies.
3. **A WAN emulator** that adds a real round-trip time and a bandwidth limit between client and server, so that measured page-load times can be compared with the textbook formulas.
4. **A caching proxy** with freshness, validation and `CONNECT` tunnels, usable from a real browser.
5. **A dashboard** on `http://localhost:8087` designed as a library reading room: requests are slips, responses are stamped, and the four strategies are drawn as connection waterfalls.

Table 1: Ports used by the HTTP module

Port	Role
TCP 8800 (and 80 when free)	Origin server: `www.cse.myuniv.edu` and `intranet.cse.myuniv.edu`
TCP 8801	WAN emulator in front of the origin (default RTT 100 ms, 4 Mbit/s)
TCP 3128	Caching proxy (set it as a browser's HTTP proxy)
HTTP 8087	Dashboard



Figure 1: The HTTP dashboard.

2 Message Formats

```
GET /news.html HTTP/1.1                  request line: method, target, version
Host: www.cse.myuniv.edu                 mandatory in HTTP/1.1 (virtual hosting)
User-Agent: ju-client/1.0
If-None-Match: "news-1"                  conditional: "only if it changed"
                                         empty line
HTTP/1.1 304 Not Modified                status line: version, code, reason
Date: Thu, 12 Nov 2026 10:15:02 GMT
ETag: "news-1"
Cache-Control: max-age=30
                                         empty line; a 304 never has a body
```

Listing 1: A request and its response. Every line ends in CRLF; an empty line ends the header section.

The length of a body is given either by `Content-Length` or by **chunked** transfer coding, where each chunk is its size in hexadecimal, `CRLF`, the data and `CRLF`, and a chunk of size 0 ends the body. Chunking lets a server start sending before it knows the total length.

Table 2: Status-code classes, with codes produced by the server

Class	Examples
1xx Informational	100 Continue
2xx Success	200 OK, 201 Created, 204 No Content, 206 Partial Content
3xx Redirection	301, 302, 303 See Other (after POST), 304 Not Modified
4xx Client error	400 (no Host), 401, 403, 404, 405, 411, 416
5xx Server error	500, 501 (unknown method), 505 (version not supported)

3 System Architecture

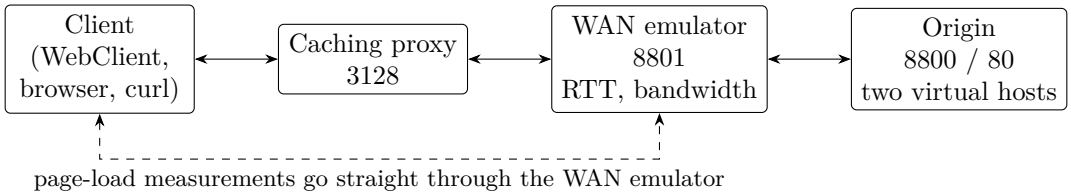


Figure 2: Components of the HTTP module.

Table 3: Source files

File	Responsibility
`core/Msg.java`	One HTTP/1.x message: start line, header fields, body framing (Content-Length, chunked, or read until close).
`server/Origin.java`	One thread per TCP connection; answers requests strictly in order; keep-alive timeout, `Connection: close`, HTTP/1.0 rules.
`server/Site.java`	The virtual hosts and every feature above.
`link/Link.java`	WAN emulator: holds the first bytes for 1 RTT (the handshake), delays every chunk by RTT/2 each way, serialises at the link rate.
`client/WebClient.java`, `PageLoad.java`	Client and the four page-load strategies with their formulas.
`proxy/Proxy.java`	Freshness, validation, `no-store`/`private`, `CONNECT` tunnels.

4 Implementation

4.1 Connections: Persistence, Pipelining and Head-of-Line Blocking

The origin server reads requests from a connection one after another and writes each response before reading the next request. A client may still write several requests at once: they wait in the TCP receive buffer and are answered **in order**. This is what makes pipelining work, and it is also the cause of **head-of-line blocking**: a slow first response delays every response behind it. The connection is kept open if the request asks for it (HTTP/1.1 by default, HTTP/1.0 only with `Connection: keep-alive`) and closed otherwise.

4.2 Page-Load Strategies and Their Formulas

For a base page with N referenced objects, ignoring transmission time, the textbook gives:

Table 4: Page-load time for a base file plus N objects

Strategy	Connections	Time
Non-persistent	$N+1$	$(N+1) \times 2\,\mathrm{RTT}$
Non-persistent, k parallel	$N+1$	$2\,\mathrm{RTT} + \lceil N/k \rceil \times 2\,\mathrm{RTT}$
Persistent (keep-alive)	1	$2\,\mathrm{RTT} + N \times \mathrm{RTT}$
Persistent + pipelining	1	$2\,\mathrm{RTT} + 1\,\mathrm{RTT} = 3\,\mathrm{RTT}$

```
switch (mode) {
    case NONPERSISTENT: res.theoryRtts = 2.0 * (n + 1);
                        res.formula = "(N+1) x 2.RTT"; break;
    case PARALLEL:      res.theoryRtts = 2 + 2.0 * Math.ceil(n / (double) parallel);
                        res.formula = "2.RTT + ceil(N/k) x 2.RTT"; break;
    case PERSISTENT:    res.theoryRtts = 2 + n;
                        res.formula = "2.RTT + N x RTT"; break;
    default:            res.theoryRtts = 3;
                        res.formula = "2.RTT + 1 RTT = 3 RTTs";
}
res.theoryMs = res.theoryRtts * rtt + txMs;
```

Listing 2: The formulas used for comparison (`client/PageLoad.java`); transmission time at the link rate is added.

4.3 The Caching Proxy

For a cacheable GET the proxy looks up the URL. A **fresh** copy is answered locally (**HIT**). A **stale** copy is **revalidated**: the request is sent to the origin with `If-None-Match` and `If-Modified-Since`, and a `304` reply lets the proxy reuse its stored body (**REVALIDATED**). Otherwise the response is fetched and stored if it is a cacheable 200 (**MISS**). The freshness lifetime comes from `max-age`, else `Expires`, else 10% of the time since `Last-Modified` (the RFC 9111 heuristic). Responses with `Set-Cookie`, `no-store` or `private`, and requests with `Authorization`, are never stored.

```
Entry e = cacheableReq ? cache.get(key) : null;
if (e != null && e.fresh() && !clientNoCache) {                        // HIT
    e.hits++;
    return copy(e.resp).set("Age", e.ageMs() / 1000).set("X-Cache", "HIT").set("Via", "1.1
    ju-proxy");
}
boolean conditional = false;
if (e != null) {                                                       // stale: revalidate
    instead of refetching
    if (e.resp.has("ETag"))          { up.set("If-None-Match", e.resp.get("ETag"));
    conditional = true; }
    if (e.resp.has("Last-Modified")) { up.set("If-Modified-Since", e.resp.get("Last-Modified
    ")); conditional = true; }
}
Msg r = fetch(host, port, up);
if (conditional && r.status() == 304 && e != null) {                   // REVALIDATED
    e.storedAt = System.currentTimeMillis(); e.freshMs = freshness(r, e.freshMs);
    return copy(e.resp).set("Age", 0).set("X-Cache", "REVALIDATED").set("Via", "1.1 ju-proxy
    ");
}
long fresh = freshness(r, -1);                                         // MISS
boolean store = cacheableReq && r.status() == 200 && fresh > 0 && !r.has("Set-Cookie")
        && !r.headerHas("Cache-Control", "no-store") && !r.headerHas("Cache-Control", "
    private");
if (store) cache.put(key, new Entry(key, r, fresh));

/** Freshness lifetime: max-age, else Expires, else 10% of the time since Last-Modified. */
static long freshness(Msg r, long fallback) {
    String cc = r.get("Cache-Control");
    if (cc != null) {
        if (cc.contains("no-cache")) return 0;
        for (String p : cc.split(",")) { p = p.trim(); if (p.startsWith("max-age=")) return
    Long.parseLong(p.substring(8)) * 1000; }
    }
```

```
28      if (r.has("Expires")) { long x = Msg.parseDate(r.get("Expires")); if (x > 0) return Math
        .max(0, x - System.currentTimeMillis()); }
29      if (r.has("Last-Modified")) { long lm = Msg.parseDate(r.get("Last-Modified")); if (lm >
        0) return Math.min(60_000, (System.currentTimeMillis() - lm) / 10); }
30      return fallback;
31  }
```

Listing 3: HIT, revalidation and MISS in `proxy/Proxy.java` (abridged).

The proxy also answers `CONNECT` by opening a TCP tunnel, so HTTPS sites still work through it, although their content cannot be cached. From a real browser with the proxy set to `127.0.0.1:3128`, `http://www.cse.myuniv.edu/gallery.html` is served from the lab server: the first load is a MISS and the second a HIT.

4.4 State, Authentication and Other Features

- **Cookies:** `POST /login` returns `303 See Other` with `Set-Cookie: sid=...; HttpOnly`. The client sends `Cookie: sid=...` with every later request, and the server looks the session up: the state lives on the server, the cookie is only its key.
- **Basic authentication:** `/private/` answers `401` with `WWW-Authenticate: Basic`; the retry carries `Authorization: Basic YWxpY2U6YWxpY2UxMjM=`, which is only base64 of `alice:alice123`: encoding, not encryption.
- **Virtual hosting:** the same IP and port serve two sites chosen by the `Host` header; an HTTP/1.1 request without `Host` is a `400`.
- **Ranges:** `Range: bytes=1000-1999` returns `206 Partial Content` with `Content-Range`; a range past the end returns `416`. This is how downloads resume (compare FTP's `REST`).
- **Chunked**, **100-continue**, **gzip** (with `Vary`), redirects 301/302/303, and the methods GET, HEAD, POST, PUT (201/204), DELETE and OPTIONS.

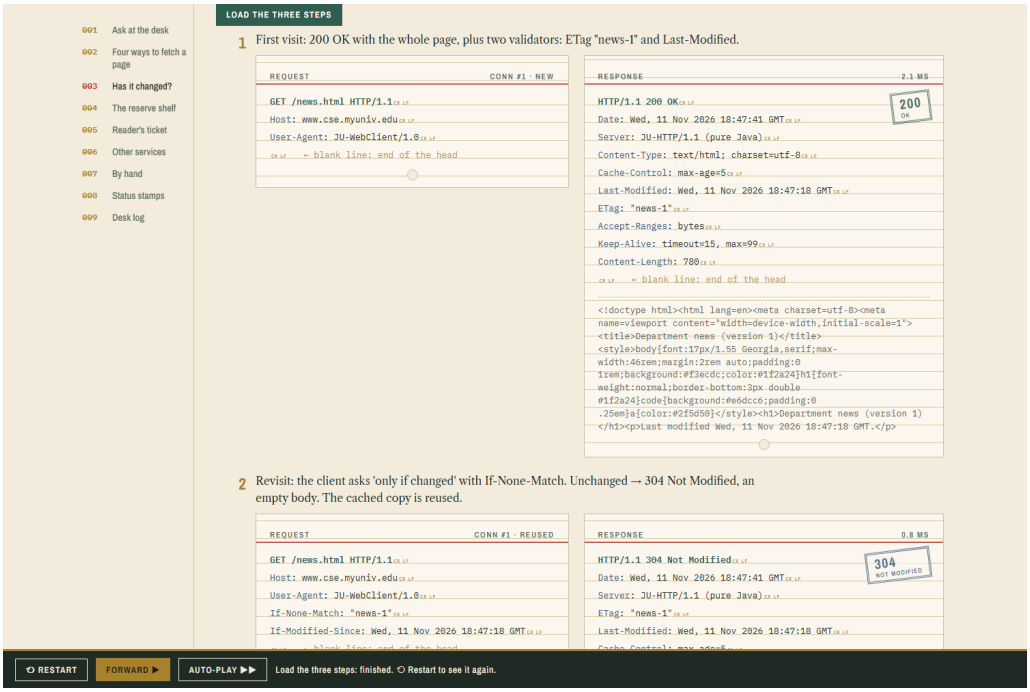


Figure 3: Conditional GET: 200 with an ETag, then 304 with no body, then 200 with a new ETag after the page changed.

5 Experiments, Verification and Results

5.1 Running the Module

```
cd HTTP
run.bat                  # origin, WAN link, proxy and dashboard (http://localhost:8087)
run.bat --verify         # 52 automated checks
python interop_test.py   # Python http.client / urllib and curl against our server and proxy
      (11 checks)

curl -v http://localhost:8800/
curl -x 127.0.0.1:3128 http://www.cse.myuniv.edu/news.html     # through the caching proxy
```

Listing 4: Starting the servers, running the checks, and testing with curl.

5.2 Page-Load Measurements

The page `gallery.html` references 10 images ($N = 10$, 11 objects). With RTT = 100 ms, $k = 5$ and a 4 Mbit/s link, all four strategies were run for real through the WAN emulator:

Table 5: Measured page-load times against the formulas (RTT 100 ms, 4 Mbit/s)

Strategy	**TCP connections**	**Predicted**	**Measured**
Non-persistent (HTTP/1.0)	11	2287 ms	2472 ms
Non-persistent, 5 parallel	11	687 ms	693 ms
Persistent (keep-alive)	1	1287 ms	1459 ms
Persistent + pipelining	1	387 ms	436 ms

The predicted values include transmission time at 4 Mbit/s; the remaining difference is the server's processing and the operating system's scheduling. The ordering non-persistent > persistent > parallel > pipelined matches the theory.

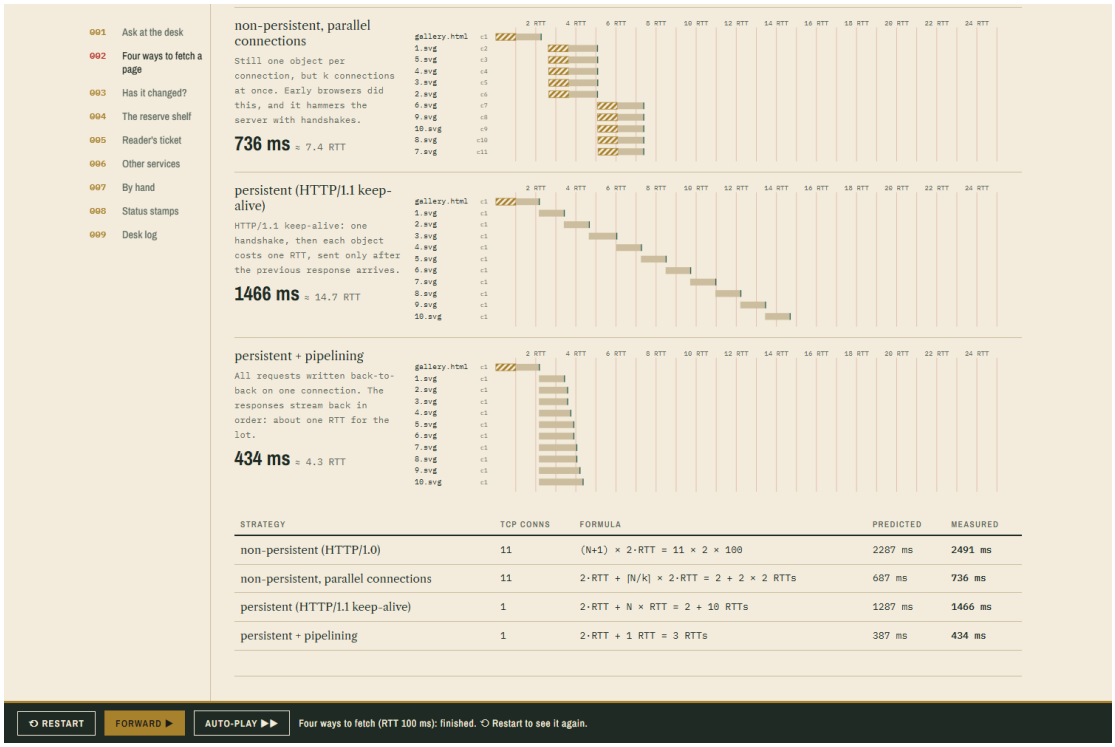


Figure 4: The four strategies drawn as connection waterfalls; hatched bars are TCP handshakes.

5.3 The Proxy

Loading the gallery through the proxy with an empty cache took about 2.4 s (11 MISSes, about 220 ms each across the WAN). The second load took about 10 ms (11 HITs, about 0.3 ms each). After every copy was made stale, the third load again took about 2.4 s but every object was REVALIDATED with a `304`: revalidation saves bytes, not round trips.

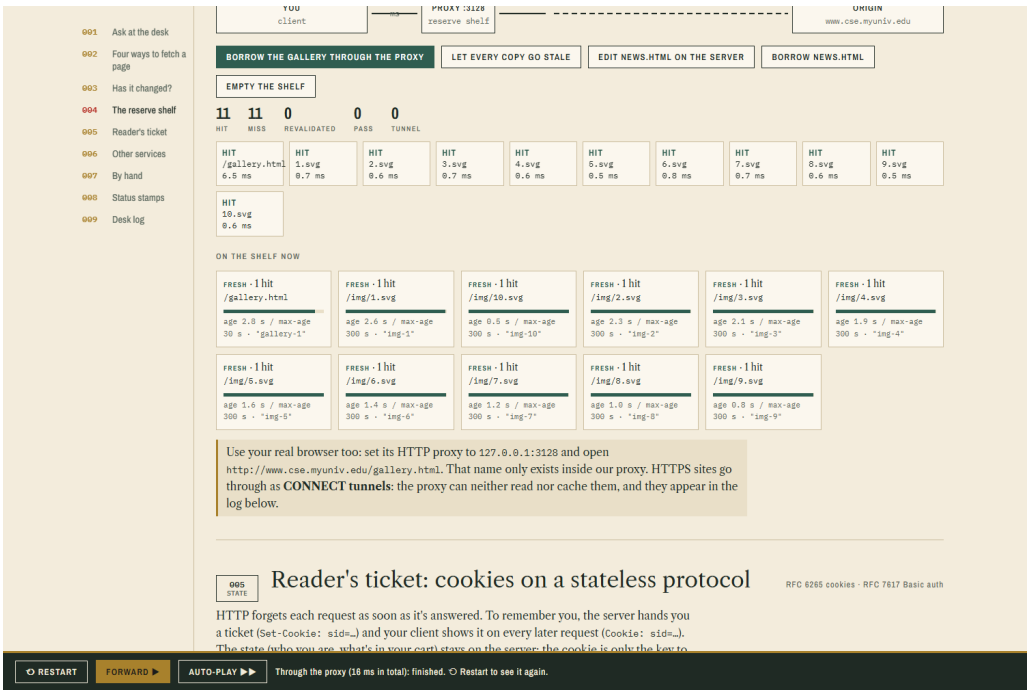


Figure 5: The proxy's reserve shelf: MISS, then HIT, then REVALIDATED.

5.4 Automated Verification

All **52 checks** of `Verify.java` and all **11** interoperation checks pass. A selection:

Table 6: Selected verification checks (all passed)

GET → 200 with a matching Content-Length; HEAD has the same headers and no body; 404, 405 + Allow, 501, 500, 505
HTTP/1.1 without Host → 400; HTTP/1.0 without Host → 200 and the connection closes; virtual hosting by Host
Three pipelined requests in one write → three responses in order; four requests on one keep-alive connection
Non-persistent uses 11 connections; each strategy within tolerance of its formula; the order of the four is correct
ETag and Last-Modified; If-None-Match and If-Modified-Since → 304; after a change → 200 with a new ETag
Range → 206 with exactly those bytes; suffix range; beyond the end → 416
gzip is smaller and decodes to the same page; PUT → 201 then 204; chunked request body stored intact
`Expect: 100-continue` → 100 then 201; `/clock` is chunked with no Content-Length; POST without length → 411
Cookies: unknown without a cookie, 303 + Set-Cookie on login, remembered afterwards; wrong password → 403
Basic authentication: 401 + WWW-Authenticate, then 200 with the right credentials
Proxy: MISS, then HIT ($\geq 5\times$ faster) with Via and Age, then 304 → REVALIDATED

6 Conclusion

1. **Round trips, not bandwidth, dominate small-object pages.** Measured through a real 100 ms link, non-persistent connections took about 2.5 s for 11 objects, keep-alive about 1.5 s and pipelining about 0.44 s, each close to its textbook formula.

2. **In-order responses explain both pipelining and head-of-line blocking.** Because a connection answers requests strictly in sequence, pipelining is possible, but one slow response delays all that follow; this motivated multiplexing in HTTP/2.

3. **Caching and validation save traffic.** A proxy HIT avoided the WAN entirely, and a conditional GET returned 304 with no body; together they explain why repeat visits to a site are fast.

4. **State is added on top of a stateless protocol.** Cookies give the server a key to its own session table, and Basic authentication shows why credentials must travel over TLS.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.

- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.